SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Brasilla R	**Reg. No.:**	192511357
Date:	28/10/2026	**Duration:**	60 minutes

Code of Conduct

I Brasilla R / 192511357 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes
Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.

Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

ARCHITECTURE DESIGN REPORT

Vehicle Service and Maintenance Management System

Problem Statement: Develop a system that allows vehicle owners to book service appointments, maintain service history, receive maintenance reminders, and track service status.

1. Analyze System Requirements

1.1 System Objectives

- To let vehicle owners book, reschedule and cancel service appointments online at any time.
- To keep a complete, secure and searchable service history for every registered vehicle.
- To send timely maintenance reminders based on date and odometer reading.
- To let owners track the live status of their vehicle from check-in to delivery.
- To help the service centre manage its workload, job cards, billing and reports efficiently.

1.2 Functional Requirements that Influence the Architecture

- **Appointment booking:** Many owners may try to book the same time slot at once, so the booking function needs consistent, transaction-safe data handling.
- **Service history:** Records must be stored permanently and retrieved quickly, which needs a reliable central database and a separate history module.
- **Maintenance reminders:** Reminders depend on due dates and mileage, so they must be generated by a scheduled background process and not by user requests.
- **Service status tracking:** Status changes by mechanics must reach owners immediately, which needs asynchronous, event-based notification.
- **External integration:** Online payment, SMS, email and push notification need well-defined interfaces to external systems.
- **Multiple user roles:** Owner, advisor, mechanic and administrator need different screens and permissions on web and mobile devices.

1.3 Non-Functional Requirements that Influence the Architecture

- **Performance:** Pages should respond within 2 to 3 seconds for about 1,000 concurrent users.
- **Security:** Authentication, role-based access, encrypted communication and protection of personal and payment data.
- **Availability and reliability:** Availability of at least 99.5 percent and no loss of bookings or records on failure.
- **Scalability:** Support for more branches, vehicles and users without redesign.
- **Maintainability:** Easy to modify, test and extend, with clear separation of responsibilities.

1.4 Key Quality Attributes

The quality attributes in order of importance for this system are availability, security, scalability, performance, reliability and maintainability. Availability and reliability matter most because owners book and track services at all hours. Security is vital because the system holds personal, vehicle and payment data. Scalability and performance are needed because load rises during peak periods such as weekends and festival seasons, and maintainability is needed because service packages, rules and integrations keep changing.

2. Select a Suitable Software Architecture

2.1 Selected Architectural Style

The selected style is a **Hybrid Architecture**. It combines a **Client-Server Layered Architecture** (presentation, API gateway, business service and data layers) with a **Microservices-based business layer** and **Event-Driven communication** for reminders and notifications.

2.2 Justification

- **Layered client-server:** The layered, client-server structure separates the web and mobile clients from the business logic and data, so each layer can be developed, tested and changed independently.

- **Microservices:** Booking, tracking, history, reminders and billing are separate business capabilities with different load patterns. As independent services they can be scaled, deployed and maintained separately, and a failure in one does not bring the whole system down.
- **Event-driven:** Maintenance reminders and status updates are naturally events (for example ReminderDue or StatusUpdated). A message broker lets the services publish events without waiting for the notification service, which keeps booking and tracking fast and makes delivery retry possible.
- **API gateway:** An API gateway gives a single secure entry point for authentication, routing and rate limiting for all clients.

2.3 Alternatives Considered

- **Monolithic layered architecture:** A single deployable unit is simple to build, but it cannot scale parts independently, and one fault can stop the entire system.
- **Pure serverless:** It is well suited for small tasks, but it makes long-running jobs and database-heavy booking transactions harder to control and can lead to vendor dependence.
- **Fully fine-grained microservices:** Splitting into too many tiny services would add unnecessary complexity for this size of project, so the services are kept at business-capability level.

3. Software Architecture Diagram

Figure 1 shows the overall structure of the proposed system. Rectangles are components, dashed rectangles are layers, and labelled arrows show the direction and the communication mechanism.

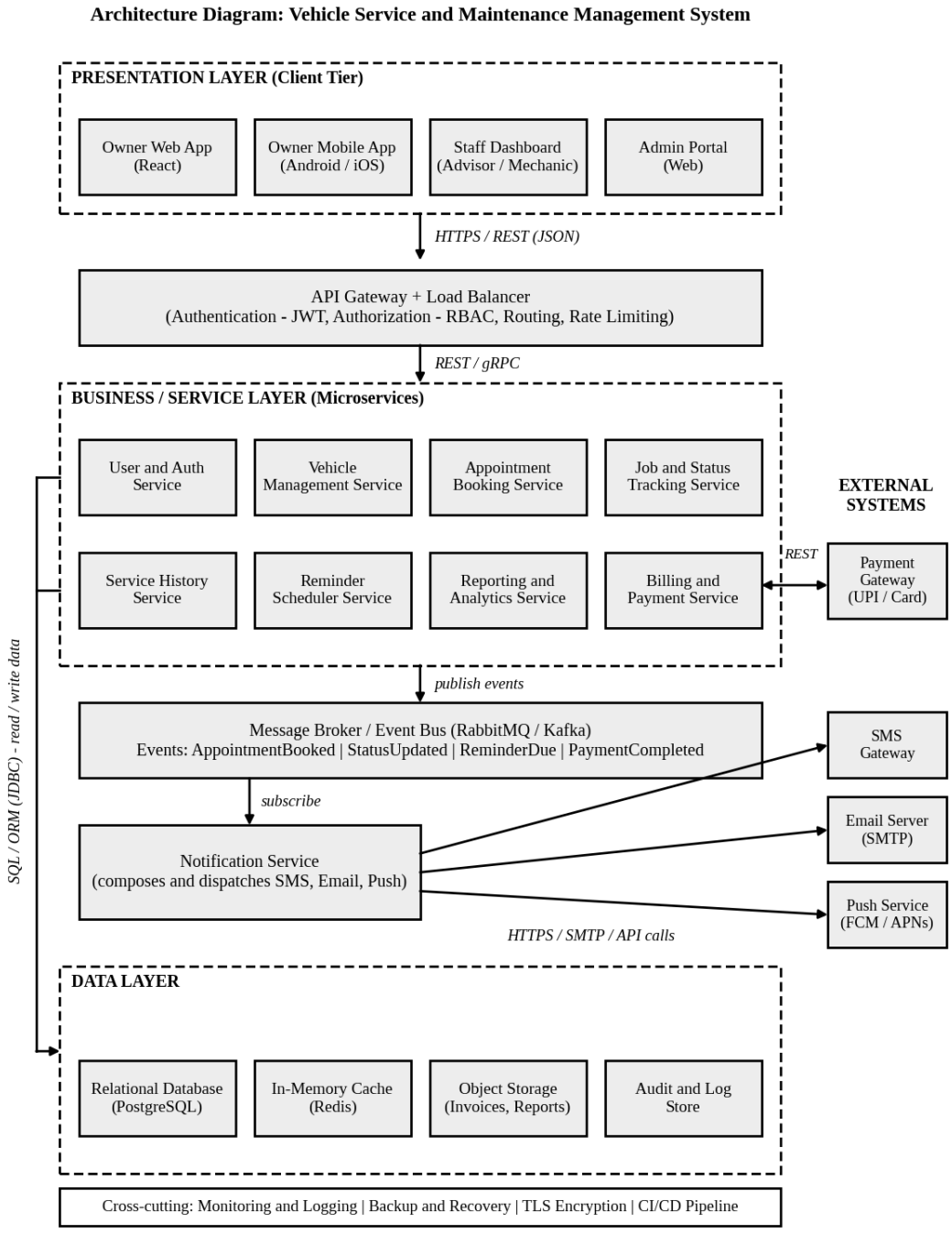


Figure 1: Software Architecture Diagram

4. Explain Components and Their Interactions

4.1 Purpose and Responsibilities of Each Component

- **Presentation layer:** Web application, mobile application, staff dashboard and admin portal. They show forms, calendars and status timelines, and send user requests to the gateway.
- **API Gateway and Load Balancer:** The single entry point. It validates the JWT token, checks role permissions, balances load, routes requests to the correct service and limits excessive requests.
- **User and Auth Service:** Registers users, verifies login by OTP or password, and manages roles and permissions.
- **Vehicle Management Service:** Stores vehicle details such as registration number, model, fuel type and odometer reading for each owner.
- **Appointment Booking Service:** Shows available slots, creates, reschedules and cancels appointments, and prevents double booking by using database transactions.
- **Job and Status Tracking Service:** Creates the job card at check-in, lets mechanics update stages (Vehicle Received, Inspection, Work in Progress, Quality Check, Completed, Delivered) and publishes a StatusUpdated event.
- **Service History Service:** Stores the work done, parts replaced, cost and odometer reading after each service, and provides search and PDF download.
- **Reminder Scheduler Service:** Calculates the next service due date and mileage, runs a daily scheduled check and publishes ReminderDue events.
- **Billing and Payment Service:** Creates invoices, connects to the payment gateway and records payment status.
- **Reporting and Analytics Service:** Generates reports on appointments, revenue, technician workload and customer feedback.
- **Notification Service:** Receives events from the broker, prepares messages and sends SMS, email and push notifications with retry on failure.

- **Message Broker (Event Bus):** Decouples services by carrying events with publish and subscribe semantics.
- **Data layer:** The relational database holds the master data, the cache speeds up frequent reads such as slot availability, the object storage holds invoices and reports, and the audit store records important actions.

4.2 Communication and Data Exchange

- **Client to gateway:** Clients communicate with the API Gateway over HTTPS using REST with JSON.
- **Gateway to services:** The gateway forwards requests to the services over REST or gRPC, and each service has its own API.
- **Service to service (events):** Services publish events to the message broker, and the Notification Service subscribes to them. Communication is asynchronous.
- **Services to data layer:** Services read and write the database using SQL through an ORM, and use the cache for hot data.
- **Services to external systems:** The Billing Service calls the payment gateway using REST over HTTPS, and the Notification Service calls SMS, SMTP and push services through their APIs.

4.3 Overall Workflow of the System

1. The owner logs in through the web or mobile application, and the gateway authenticates the request through the User and Auth Service.
2. The owner adds a vehicle, which the Vehicle Management Service saves in the database.
3. The owner selects a service and a time slot. The Appointment Booking Service checks availability in the cache and database, confirms the slot and publishes an AppointmentBooked event.
4. The Notification Service consumes the event and sends a confirmation by SMS and email.
5. On the service day, the advisor checks the vehicle in and the Job and Status Tracking Service creates a job card with an estimate for the owner's

approval.

6. The mechanic updates the status at each stage. Every update is saved and a StatusUpdated event is published, so the owner sees the progress live and receives an alert.
7. After the work is completed, the Billing Service generates the invoice and the owner pays through the payment gateway.
8. The Service History Service stores the full record, and the Reminder Scheduler Service calculates the next due date and mileage.
9. When the due date approaches, the scheduler publishes a ReminderDue event, and the Notification Service reminds the owner, who can book again with one click.
10. The administrator and management view reports from the Reporting and Analytics Service.

5. Discuss Quality Attributes

5.1 Scalability

Each microservice is stateless and can be scaled horizontally behind the load balancer, so busy services such as booking and status tracking can be scaled without scaling the whole system. The cache reduces database load, the database can use read replicas, and the message broker absorbs sudden bursts of events. New branches are added as data and configuration and not as code changes.

5.2 Security

All traffic uses HTTPS (TLS). The gateway enforces JWT authentication and role-based access control for owner, advisor, mechanic and administrator. Passwords are stored as salted hashes, sensitive data is encrypted at rest, inputs are validated against injection attacks, and the card details are never stored because payment is handled by the gateway (PCI-DSS). The audit store records important actions for traceability.

5.3 Performance

The gateway balances load, Redis caches frequently read data such as slot availability and vehicle details, the database uses indexes on vehicle, appointment and date fields, and slow work such as sending notifications and generating reports is performed asynchronously. This keeps user requests within 2 to 3 seconds.

5.4 Reliability

Booking and payment operations use database transactions to keep data consistent. The message broker stores events until they are processed, failed notifications are retried, and an alternate channel is used when one fails. Daily backups and an audit trail protect the service records from loss.

5.5 Maintainability

Each layer and service has a single responsibility and a clear interface, so a change to one service, such as new reminder rules, does not affect the others. Services can be tested separately, deployed through a CI/CD pipeline, and monitored with central logging, which makes faults easier to find and fix.

5.6 Availability

Multiple instances of every service run behind the load balancer, so the failure of one instance does not stop the service. Services are loosely coupled through the event bus, so a failure in notification does not stop booking. Database replication with automatic failover, health checks and monitoring help to reach the target availability of 99.5 percent or more.

6. Justify Architectural Decisions

6.1 Rationale Behind the Architectural Choices

- A hybrid style was chosen because no single style meets all the needs. The layered structure gives clarity, the microservices give scalability and fault isolation, and events give loose coupling for reminders and tracking.
- The API gateway was introduced to give one secure, consistent entry point for all clients.

- A relational database was selected for the core data because bookings, invoices and service records need strong consistency and relationships, while a cache and object storage handle speed and files.
- External services for payment, SMS and email are used through APIs instead of being built, which reduces cost and risk.

6.2 Trade-offs Considered

- Microservices add more deployment, monitoring and testing effort than a monolith, but they were accepted for the benefits of scaling and isolation. The services were kept at a moderate size to control this complexity.
- Event-driven communication gives eventual consistency, so a notification may arrive a few seconds after the change. This is acceptable for reminders and status alerts, while booking and payment use synchronous, transactional calls.
- The cache improves speed but may hold slightly old data, which is handled with short expiry times and cache refresh after updates.
- Using third-party gateways saves effort but creates a dependence on them, which is reduced by retries and alternate channels.

6.3 Support for Future Enhancements, Integration and Long-term Maintainability

- **Future enhancements:** New features such as pick-up and drop-off, spare parts inventory, insurance and warranty tracking, or AI-based predictive maintenance can be added as new services without changing the existing ones.
- **System integration:** Standard REST APIs and events allow easy integration with vehicle manufacturers, telematics devices, accounting software and other workshops, and the system can later be offered to many service centres.
- **Long-term maintainability:** Clear layers, independent services, automated testing and deployment, and central logging keep the system easy to understand and change over many years, and technologies can be replaced

one service at a time.

Conclusion

The hybrid architecture, which combines a layered client-server structure, microservices and event-driven communication, fully meets the functional and quality requirements of the Vehicle Service and Maintenance Management System. It is scalable, secure, reliable and easy to maintain, and it is ready for future growth and integration.